6 — Funções e modularização

Escola Tecnológica FPFtech

Curso: Técnico em Automação Industrial

Linguagem de Programação Voltada a Microcontroladores

Semana 02

Prof. Denis Moraes Guimarães

29/09/2026

6 — Funções e modularização: percurso

Ao final da aula, você deverá conseguir

Criar funções com parâmetros e retorno, dividir um programa em arquivos e reconhecer os custos de uma solução recursiva.

1. 6.1 Parâmetros, retorno e contratos.
2. 6.2 Organização modular: interface e implementação.
3. 6.3 Recursividade: caso-base e uso de memória.

Pré-requisitos: variáveis, condições, laços e vetores.
Contexto: processar temperaturas de uma máquina em C11, no computador.

Por que extrair uma função?

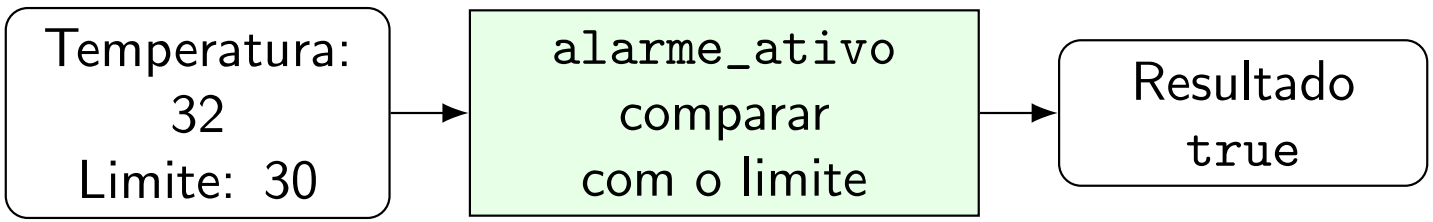
```
bool alarme_motor = temperatura_motor >= 30;
bool alarme_painel = temperatura_painel >= 30;
```

- A decisão se repete; sua regra pode mudar.
- Uma função recebe os dados e concentra a regra.
- Nomes claros explicam a intenção de cada chamada.

```
bool alarme_motor = alarme_ativo(temperatura_motor, 30);
bool alarme_painel = alarme_ativo(temperatura_painel, 30);
```

Trechos de uso; a definição de `alarme_ativo` vem a seguir.

Uma função transforma entradas em resultado



Separar decisão de ação

A função calcula se há alarme. Outra parte do programa imprime a mensagem ou, futuramente, aciona uma saída digital.

Assim, podemos testar a regra sem conectar o equipamento.

6.1 — Anatomia de uma função em C

```c
#include <stdbool.h>

bool alarme_ativo(int temperatura_c, int limite_c)
{
    return temperatura_c >= limite_c;
}
```

- `bool`: tipo do valor devolvido.
- `alarme_ativo`: nome; os parâmetros ficam entre parênteses.
- `temperatura_c` e `limite_c`: entradas locais.
- `return`: devolve um valor e encerra esta chamada.

Declaração, definição e chamada

```c
/* Declaracao: apresenta a interface. */
bool alarme_ativo(int temperatura_c, int limite_c);

/* Chamada: usa a interface com argumentos. */
bool ativo = alarme_ativo(32, 30);
```

- A **declaração** termina com ponto e vírgula.
- A **definição** inclui o corpo entre chaves.
- Na chamada, 32 e 30 são argumentos.
- Declare a função antes de usá-la; o compilador verifica os tipos.

Um protótipo `int ler(void);` explicita ausência de parâmetros em C11.

Retornar um valor ou executar uma ação

```c
int limitar(int valor, int minimo, int maximo)
{
    if (valor < minimo) return minimo;
    if (valor > maximo) return maximo;
    return valor;
}

void mostrar_inicio(void)
{
    puts("Monitor iniciado"); /* requer <stdio.h> */
}
```

Contrato: `minimo <= maximo`. Uma função `void` não fornece valor; pode terminar com `return;`.

Evite caminhos sem retorno

Toda execução de uma função que produz resultado deve devolver o valor esperado antes de ele ser usado.

C passa todos os argumentos por valor

```c
void tentar_incrementar(int contador)
{
    contador++; /* altera somente a copia local */
    printf("Local: %d\n", contador);
}

int contador = 10;
tentar_incrementar(contador);
printf("Original: %d\n", contador);
```

Rastreamento

A chamada copia 10. A cópia torna-se 11; o original continua 10.

Use retorno para fornecer um novo valor: `contador = incrementar(contador);`. Considere o intervalo do tipo.

Modificar o original: passar seu endereço

```c
void incrementar_no_local(int *contador)
{
    if (contador != NULL) {
        (*contador)++;
    }
}

int total = 10;
incrementar_no_local(&total); /* total passa a 11 */
```

- `&total`: endereço de `total`; `*contador`: objeto apontado.
- O ponteiro também é copiado; ele permite acessar o original.
- Exige endereço válido e valor menor que `INT_MAX`.

Isto é alteração indireta por ponteiro, não um modo nativo de passagem por referência em C. `NULL` vem de `<stddef.h>`.

Contratos: o que uma função promete?

Parte	Exemplo: `alarme_ativo(t, limite)`
Propósito	Indicar temperatura igual ou superior ao limite.
Entradas	Temperatura e limite inteiros, ambos em graus Celsius.
Pré-condição	Leituras já validadas pela camada de aquisição.
Pós-condição	Retorna verdadeiro se, e somente se, `t >= limite`.
Efeitos	Não altera os argumentos nem acessa GPIO.

Unidades fazem parte do contrato

Comparar Celsius com milivolts produz uma decisão sem significado, mesmo que ambos usem o tipo `int`.

Funções puras e efeitos externos

Decisão pura

Mesmas entradas produzem o mesmo retorno; nenhum estado externo é alterado.

Exemplos: comparar uma leitura, calcular uma média, limitar um valor.

Ação com efeito

Interage com entrada/saída ou modifica um objeto externo.

Exemplos: imprimir, ler um botão, alterar um contador por ponteiro.

Separar cálculo e entrada/saída permite reutilizar a lógica e montar testes com dados conhecidos.

Escopo e duração das variáveis locais

```
int dobro(int entrada)
{
    int resultado = 2 * entrada;
    return resultado;
}
```

- `entrada` e `resultado` pertencem a esta chamada.
- Os nomes não podem ser usados fora de seu escopo.
- Variáveis locais automáticas deixam de existir ao retornar.
- Retornar o **valor** é válido; retornar o endereço de uma variável local automática cria um ponteiro inválido.

Pré-condição do exemplo: o dobro deve caber no tipo `int`. Variáveis globais dificultam acompanhar quem altera o estado.

Vetores: dados e quantidade viajam juntos

```c
int maior_leitura(const int valores[], size_t quantidade)
{
    int maior = valores[0];
    for (size_t i = 1; i < quantidade; ++i) {
        if (valores[i] > maior) maior = valores[i];
    }
    return maior;
}
```

`size_t` vem de `<stddef.h>`.

- Contrato: `quantidade > 0`; vetor com elementos suficientes.
- No parâmetro, `valores[]` é ajustado para ponteiro.
- `const` impede modificar os elementos por este parâmetro.

O tamanho deve ser calculado no chamador

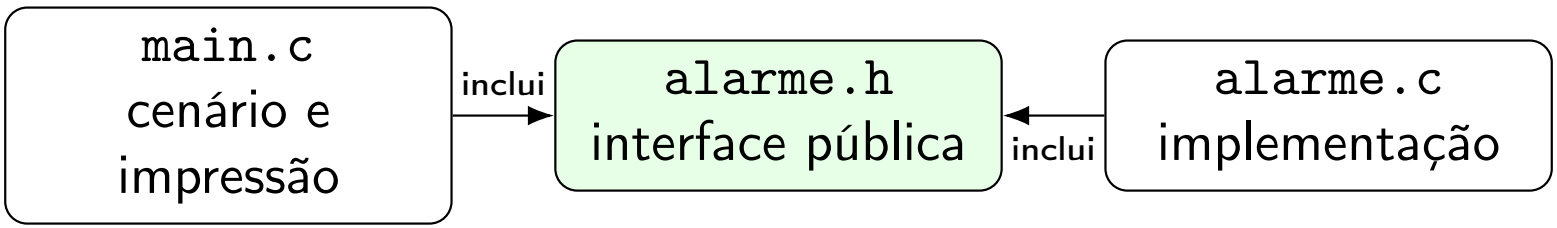
```c
int leituras[] = {28, 31, 29, 30};
size_t n = sizeof leituras / sizeof leituras[0];
int maior = maior_leitura(leituras, n);
printf("Maior leitura: %d C\n", maior); /* 31 C */
```

Armadilha comum

Dentro de `maior_leitura`, `sizeof valores` mede o ponteiro, não o vetor original. A função precisa receber a quantidade.

Antes de chamar com dados externos, valide se o vetor não está vazio. A função não descobre sozinha quantos elementos são válidos.

6.2 — Organização modular do monitor



- Cada arquivo tem uma responsabilidade identificável.
- O cabeçalho informa como usar o módulo.
- O arquivo `.c` contém a implementação da regra.
- A interface permite trocar a implementação preservando o uso.

Cabeçalho: interface e proteção de inclusão

```c
/* alarme.h */
#ifndef ALARME_H
#define ALARME_H

#include <stdbool.h>

/* Unidades: graus Celsius. Igualdade ativa alarme. */
bool alarme_ativo(int temperatura_c, int limite_c);

#endif
```

- As guardas evitam processar o conteúdo repetidamente na mesma unidade de tradução.
- O cabeçalho inclui o tipo de que sua interface depende.
- Neste projeto, corpos de funções ficam no `.c`.

Implementação: uma regra em um lugar

```c
/* alarme.c */
#include "alarme.h"

bool alarme_ativo(int temperatura_c, int limite_c)
{
    return temperatura_c >= limite_c;
}
```

Por que incluir o próprio cabeçalho?

O compilador pode detectar divergências entre a interface declarada e a definição da função.

As aspas identificam o cabeçalho do projeto. As bibliotecas padrão, como `<stdio.h>`, usam sinais de menor e maior.

Programa principal: orquestrar as chamadas

```c
#include <stdio.h>
#include "alarme.h"

int main(void)
{
    const int leituras[] = {28, 30, 32};
    const size_t n = sizeof leituras / sizeof leituras[0];
    for (size_t i = 0; i < n; ++i) {
        bool ativo = alarme_ativo(leituras[i], 30);
        printf("%d C: %s\n", leituras[i],
               ativo ? "ALARME" : "normal");
    }
    return 0;
}
```

A simulação inclui uma leitura exatamente no limite.

Compilar e ligar os módulos

```
gcc -std=c11 -Wall -Wextra -Wpedantic \
    main.c alarme.c -o monitor
./monitor
```

```
28 C: normal
30 C: ALARME
32 C: ALARME
```

- Cada `.c` é compilado; o ligador resolve as referências.
- Informe os dois arquivos `.c` ao comando.
- Inclua `alarme.h`; não inclua `alarme.c` no principal.

Execute na pasta `exemplos/modular` desta aula.

Erros comuns ao criar módulos

Sintoma	O que investigar
Declaração implícita	Protótipo ausente ou cabeçalho não incluído.
Tipos conflitantes	Declaração e definição usam assinaturas diferentes.
Referência indefinida	Implementação ausente do comando de ligação.
Definição múltipla	Função definida em mais de um arquivo do projeto.
Resultado inesperado	Unidades, ordem dos argumentos e regra do limite.

Compilar sem avisos ajuda; ainda é necessário testar a regra de negócio.

6.3 — O que é recursividade?

Definição

Uma função recursiva resolve parte do problema chamando a si mesma com um problema menor.

1. **Caso-base**: resultado conhecido, sem nova chamada.
2. **Passo recursivo**: reduzir o problema.
3. **Progresso**: garantir que as chamadas alcancem o caso-base.

Exemplo didático: $S(0) = 0$ e $S(n) = n + S(n-1)$ para inteiros $n > 0$. Na aula, limitaremos n a 20.

Exemplo recursivo com domínio limitado

```c
/* Contrato: 0 <= n <= 20. */
unsigned soma_recursiva(unsigned n)
{
    if (n == 0U) return 0U; /* caso-base */
    return n + soma_recursiva(n - 1U);
}
```

- O teste ocorre antes da subtração: não há redução abaixo de zero.
- Para 20, o resultado é 210 e cabe em `unsigned`.
- O chamador deve verificar o limite antes da chamada.
- Há no máximo 21 chamadas ativas, incluindo o caso-base.

O arquivo de exemplo usa valores pequenos e conhecidos.

Rastreamento: chamadas descem, retornos sobem

Chamada	Aguarda	Depois devolve
`soma_recursiva(3)`	$3 + S(2)$	$3 + 3 = 6$
`soma_recursiva(2)`	$2 + S(1)$	$2 + 1 = 3$
`soma_recursiva(1)`	$1 + S(0)$	$1 + 0 = 1$
`soma_recursiva(0)`	Caso-base	0

Memória das chamadas

Cada chamada pode precisar guardar argumentos, estado e endereço de retorno na pilha. A profundidade cresce com n.

O tamanho exato depende da arquitetura e do compilador. Não presuma que uma otimização removerá essas chamadas.

Alternativa iterativa e custo em microcontroladores

```
unsigned soma_iterativa(unsigned n)
{
    unsigned total = 0U;
    for (unsigned i = 1U; i <= n; ++i) total += i;
    return total;
}
```

Mesmo contrato: `n <= 20`.

- Ambas executam trabalho proporcional a n neste exemplo.
- A versão iterativa usa quantidade constante de variáveis locais.
- A recursão pode esgotar uma pilha pequena; sua profundidade precisa ser analisada.
- Em firmware simples, um laço costuma facilitar prever memória e tempo.

Prática guiada — Monitor modular

1. Abra `exemplos/modular` e identifique os três arquivos.
2. Compile e confira as três linhas de saída previstas.
3. Troque o limite de 30 para 31 no principal e preveja a saída.
4. Acrescente a leitura 31 ao vetor e rode novamente.
5. Restaure o cenário inicial; explique onde a regra é implementada.

Entrega

Anote o comando usado, a saída com limite 31 e a responsabilidade de cada arquivo.

Antes de executar cada alteração, registre sua previsão.

Exercícios — Projetar antes de programar

1. Crie `bool dentro_faixa(int v, int min, int max)` com limites inclusivos. Declare a pré-condição.
2. Para o vetor `{28, 31, 29, 30}`, chame `maior_leitura` e explique por que deve passar o tamanho.
3. Rastreie `soma_recursiva(4)` e conte as chamadas, incluindo o caso-base.
4. Explique o que muda no original ao passar um inteiro ou passar seu endereço.

Desafio: crie um cabeçalho para `dentro_faixa` e mantenha a implementação em um arquivo separado.

Conferência dos exercícios

```
bool dentro_faixa(int v, int min, int max)
{
    return v >= min && v <= max;
}
```

- Contrato: `min <= max`; limites incluídos.
- Maior leitura: 31. A função recebe ponteiro e quantidade; o ponteiro não carrega o tamanho do vetor.
- $S(4) = 4 + 3 + 2 + 1 + 0 = 10$: cinco chamadas.
- Um inteiro é copiado; um endereço copiado permite alterar o objeto apontado, se válido.

Prática com limite 31 e vetor `{28,30,32,31}`: normal, normal, ALARME, ALARME.

Síntese — Uma função precisa de uma interface clara

- Parâmetros descrevem entradas; retorno descreve o resultado.
- C usa passagem por valor, inclusive para ponteiros.
- Vetores exigem contrato de tamanho e acesso válido.
- Cabeçalhos declaram interfaces; arquivos `.c` as implementam.
- Recursão exige caso-base, progresso e análise da pilha.

Verificação de saída

Você consegue explicar a regra de alarme e testar seus limites sem um sensor físico?

Próxima aula: transformar essa interface em casos de teste e investigar falhas com método.